Confiabilidade de sistemas de software distribuídos a partir de sinais OpenTelemetry

Felippe Veloso Marinho Josoe Santos Queiroz Maria Eduarda Sampaio
Matheus Amorim Soares Salles Raphael Lucas Silva Amaral

2026

Resumo

Este trabalho propõe o estudo da confiabilidade de sistemas de software distribuídos utilizando sinais coletados pelo padrão aberto OpenTelemetry. A iniciativa visa democratizar práticas de *Site Reliability Engineering* (SRE) para médias e pequenas empresas brasileiras, reduzindo a dependência de fornecedores proprietários de monitoramento.

1 Introdução

O tema de confiabilidade de software é relativamente recente, mas amplamente discutido. A ubiquidade de sistemas de software em diversas áreas da sociedade tornou-os complexos e mais críticos. Nas décadas mais recentes, com os avanços no campo da internet e sistemas de software dependentes dela, tornou-se prática comum separar diferentes partes complexas de aplicações em servidores conectados por redes (FOWLER, 2002), isto é o que denominamos sistemas distribuídos. A partir dessa definição, cada servidor em um sistema distribuído é tratado como um componente que pode falhar ou se comportar fora do esperado, similarmente a componentes de outros tipos de artefatos de engenharia.

A prática voltada à confiabilidade de sistemas web é conhecida como *Site Reliability Engineering* (SRE) (BEYER et al., 2016) e é prática comum no Brasil desde a metade da última década. Contudo, esse campo ainda não é consistentemente aplicado em empresas brasileiras, porque o tema demanda alto investimento e planejamento estratégico de longo prazo, tornando-se inviável para médias e pequenas empresas. Os custos relacionados a tempo, pessoas e fornecedores são altos demais para uma adoção da prática que realmente impacte positivamente a confiabilidade de sistemas novos.

Normalmente, um dos fornecedores mais caros da SRE é o fornecedor de monitoramento. Essas empresas têm como maior produto uma suíte de bibliotecas e ferramentas de nuvem especializadas em coletar e processar dados de monitoramento de sistemas distribuídos. Contudo, o número relativamente pequeno de fornecedores competitivos leva a preços altos e ao efeito de *vendor lock-in*, que é a incapacitação de uma organização de substituir o fornecedor por ser complexo ou caro demais. Essa situação irrita o mercado de software há anos e, para tanto, foi criado o OpenTelemetry (OpenTelemetry Authors, 2026), uma iniciativa de código aberto que tem como finalidade padronizar e simplificar a adoção de software de monitoramento.

2 Descrição do sistema

Conceitualmente, a proposta mede uma requisição que parte do front-end, atravessa uma ou mais APIs e chega a consultas de banco. Vários serviços participam do mesmo fluxo. As medidas básicas de HTTP, de banco e de uso de CPU e memória seguem a especificação do OpenTelemetry. Um servidor instrumentado entrega esses dados no mesmo formato, qualquer que seja a finalidade. Um servidor de jogo é um caso possível. Os exemplos com que o grupo já trabalha são dois, e os dois são clínicos.

O primeiro, já formalizado, é uma aplicação web em operação no atendimento clínico. A rota de prontuário é um dos caminhos. Quando o banco deixa de responder, as sessões são encerradas por segurança e as respostas HTTP 401 sobem. O banco não é medido diretamente. Parte desses 401 é autenticação normal. No modo de tempo, a resposta percebida como rápida fica abaixo de 200 ms. O segundo é o Sabiá (Azel), aplicação web em teste por psicólogos, também com front-end, API e banco, no apoio ao registro da sessão clínica.

O estudo não se apoia agora num único exemplo. Nos dois há dado clínico, e nos dois ainda não há confirmação de que esse dado possa ser liberado, nem de quais campos entrariam. Se nenhuma autorização se confirmar, outro servidor instrumentado ocupa o lugar, com o código de erro que representar a falha percebida nele.

Os dados que forem liberados chegam mascarados, com apoio de ferramentas já disponíveis. Entram código de estado, rota agregada, duração e, quando existirem, as medidas de máquina. Identidade e conteúdo clínico ficam de fora. A coleta prevista é de cerca de quinze dias.

3 Revisão de Literatura

A avaliação da confiabilidade de serviços deve considerar as falhas percebidas pelos usuários, e não apenas aquelas observadas em componentes isolados. Oppenheimer, Ganapathi e Patterson (2003) analisaram relatórios de falhas de três grandes serviços de Internet, distinguindo falhas de componentes de falhas de serviço, e identificaram o erro de operador como a maior causa em dois deles. De forma complementar, Tian, Rudraraju e Li (2004) definem falhas web como a incapacidade de obter ou entregar corretamente as informações solicitadas. A partir de registros de acesso e de erros dos servidores, os autores apontam que contar falhas não basta quando a carga varia e relacionam as falhas à carga de trabalho, aplicando o modelo de Nelson, $R = 1 - f/n$, em que f representa o número de falhas e n o número de requisições. Ao tratar como falha o erro de arquivo inexistente, equivalente ao código HTTP 404, obtiveram confiabilidades por requisição de 96,2% e 94,2% nos dois sites analisados.

Schroeder e Gibson (2006), por sua vez, investigaram aproximadamente 23 mil falhas em 22 sistemas de computação de alto desempenho, ajustando distribuições estatísticas por máxima verossimilhança aos intervalos entre falhas e aos tempos de reparo. Os resultados indicaram bom ajuste das distribuições Weibull e gama para os intervalos entre falhas, ao contrário da exponencial, enquanto a lognormal representou melhor os tempos de reparo. Na perspectiva prática, o material do Google sobre *Site Reliability Engineering* define indicadores (SLI) e objetivos (SLO) de nível de serviço, recomendando que o objetivo não seja escolhido com base no desempenho atual (JONES et al., 2016), e trata a proporção de requisições bem-sucedidas como uma aproximação razoável da indisponibilidade não planejada (ALVIDREZ, 2016).

Entre os trabalhos revisados, Tian, Rudraraju e Li calculam índices a partir de logs, mas não ajustam distribuições de tempo até a falha; Schroeder e Gibson ajustam distribuições aos tempos entre falhas, mas não partem de logs HTTP; e Oppenheimer, Ganapathi e Patterson analisam causas, sem modelagem estatística dos tempos. Esta proposta busca combinar essas abordagens, estimando índices de confiabilidade e distribuições de tempo até a falha a partir de dados de sistemas web instrumentados com OpenTelemetry, com a falha definida por um objetivo de nível de serviço declarado.

4 Metodologia

O objeto é um sistema web genérico, em que a requisição parte do front-end, atravessa uma ou mais APIs e chega ao banco de dados. A análise mede o serviço do ponto de vista do usuário e cada componente do fluxo, e uma árvore de falhas combina os índices dos componentes para prever o do serviço.

Coleta. Os componentes são instrumentados com o OpenTelemetry, cujas convenções semânticas padronizam as medidas de HTTP e de banco de dados em qualquer sistema (OpenTelemetry Authors, 2026). A métrica `http.server.request.duration` registra código de resposta, rota e duração de cada requisição. Os *spans* de cliente de banco registram as consultas feitas pela API. Quando o banco não é instrumentado, sua falha é inferida pelo sinal que ela produz na API. O Grafana exibe o mapa de chamadas entre serviços (*service graph*), e um *plugin* desenvolvido pelo grupo calcula os índices por janela. As contagens exportadas em CSV para uma planilha servem para conferir os valores do *plugin*. A coleta usa tráfego real durante aproximadamente 15 dias, condicionada à autorização do responsável por cada fonte. Os dados chegam mascarados: só entram código de resposta, rota agregada e duração.

Critério de falha. A falha é medida do ponto de vista do usuário, como falha do serviço, e não só do componente (OPPENHEIMER; GANAPATHI; PATTERSON, 2003). Há dois modos de falha. No modo *erro*, a requisição recebe código 5xx ou o código que a fonte usa para sinalizar falha percebida (por exemplo, 401 em massa quando a perda do banco encerra sessões). Desse código, conta só o excesso sobre a taxa típica da rota, porque parte dele é uso normal (GOSEVA-POPSTOJANOVA; MAZIMDAR; SINGH, 2004). No modo *lentidão*, a duração passa de $\tau = 200$ ms. O tempo é dividido em janelas de $\Delta = 5$ min, e só entram janelas com pelo menos 100 requisições, para que uma única falha não derrube o índice. Os SLOs são fixados antes da coleta, sem usar o desempenho observado (JONES et al., 2016): $R^*_{\text{erro}} = 99\%$ e $R^*_{\text{lat}} = 95\%$.

I1 — confiabilidade por requisição. Pelo modelo de Nelson, usado por Tian, Rudraraju e Li (2004) e Goseva-Popstojanova, Mazimdar e Singh (2004), $R = 1 - f/n$, em que f é o número de requisições com falha e n o total de requisições. R é a probabilidade de uma requisição ser atendida corretamente. A razão normaliza as falhas pela carga (TIAN; RUDRARAJU; LI, 2004) e aproxima a indisponibilidade percebida pelo usuário (ALVIDREZ, 2016). O índice é calculado para cada modo de falha (R_{erro} e R_{lat}) e para os dois juntos, no período inteiro, por janela (R_j) e por componente (R_i). Os percentis 95 e 99 da latência complementam R_{lat} como medida de variabilidade.

I2 — disponibilidade em relação ao SLO. A janela j está em falha quando $R_{\text{erro},j} < R^*_{\text{erro}}$ ou $R_{\text{lat},j} < R^*_{\text{lat}}$. A disponibilidade A é a fração de janelas dentro do SLO.

I3 — MTBF e MTTR. Uma sequência de janelas consecutivas em falha forma um episódio de falha. O MTBF é o tempo total dentro do SLO dividido pelo número de episódios, e o MTTR é a duração média dos episódios (O'CONNOR; KLEYNER, 2012). Como $A = \text{MTBF}/(\text{MTBF} + \text{MTTR})$, os dois mostram se o serviço perde disponibilidade por falhar com frequência ou por demorar a se recuperar.

Árvore de falhas. O evento topo é o serviço fora do SLO em uma janela (RUIJTERS; STOELINGA, 2015). A estrutura da árvore vem do *service graph*. As camadas

que a requisição atravessa em sequência (por exemplo front-end, API e banco) entram em uma porta OU, porque a falha de qualquer uma derruba o serviço. As instâncias redundantes de uma mesma camada entram em uma porta E, porque a camada só falha se todas falharem juntas (O'CONNOR; KLEYNER, 2012). Com p_i igual à fração de janelas em que a instância i falhou, e supondo falhas independentes, a probabilidade prevista de falha do serviço é

$$P_{\text{prev}} = 1 - \prod_{c} \Big(1 - \prod_{i \in c} p_i\Big), \tag{1}$$

em que c percorre as camadas e i as instâncias de cada camada. P_{prev} é comparado com a fração observada de janelas em falha, $1 - A$. Se os dois valores forem próximos, a hipótese de independência se sustenta. Se o observado for maior, há uma causa comum afetando vários componentes ao mesmo tempo, como uma implantação com defeito, que pode ser identificada pela versão do serviço registrada na telemetria.

Declaração de uso de IA. Ferramenta: Claude Opus 5.5 (Anthropic), em novembro de 2026. Finalidade: busca de referências, discussão de índices e rascunho desta seção. Estratégia: o grupo revisou e reescreveu o texto e conferiu as referências nas fontes originais. Responsabilidade: o conteúdo é de responsabilidade exclusiva dos autores.

5 Resultados Esperados

Espera-se como resultado deste trabalho:

- Um *plugin* do Grafana que calcula, a partir das métricas padronizadas pelo OpenTelemetry, a confiabilidade por requisição (R_{erro} e R_{lat}), a disponibilidade em relação ao SLO, o MTBF e o MTTR;
- A generalização do *plugin*: qualquer sistema que exporte métricas convertíveis em contagens de sucesso, falha e duração passa a ter os mesmos índices de confiabilidade, levando a percepção do usuário para além do sistema estudado;
- Os valores desses índices para o sistema estudado nos cerca de quinze dias de coleta, separando a parcela devida a erro da devida a lentidão;
- A árvore de falhas do sistema, com os pontos únicos de falha e a comparação entre a falha prevista a partir dos componentes e a observada no serviço, indicando se há causa comum;
- Índices que reflitam a percepção de usuários e engenheiros: os episódios de falha detectados devem coincidir com os incidentes relatados no período.

Referências

ALVIDREZ, M. Embracing risk. In: BEYER, B. et al. (Ed.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. cap. 3. Disponível em: <https://sre.google/sre-book/embracing-risk/>. Acesso em: 6 nov. 2026.

BEYER, B. et al. (Ed.). *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O'Reilly Media, 2016.

FOWLER, M. *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002.

GOSEVA-POPSTOJANOVA, K.; MAZIMDAR, S.; SINGH, A. D. Empirical study of session-based workload and reliability for web servers. In: *Proceedings of the 15th International Symposium on Software Reliability Engineering (ISSRE)*. Saint-Malo, France: [s.n.], 2004.

JONES, C. et al. Service level objectives. In: BEYER, B. et al. (Ed.). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016. cap. 4. Disponível em: <https://sre.google/sre-book/service-level-objectives/>. Acesso em: 6 nov. 2026.

O'CONNOR, P. D. T.; KLEYNER, A. *Practical Reliability Engineering*. 5. ed. Chichester: John Wiley & Sons, 2012.

OpenTelemetry Authors. *OpenTelemetry*. 2026. Disponível em: <https://opentelemetry.io/docs/concepts/signals/>. Acesso em: 6 nov. 2026.

OPPENHEIMER, D.; GANAPATHI, A.; PATTERSON, D. A. Why do internet services fail, and what can be done about it? In: *Proceedings of the 4th USENIX Symposium on Internet Technologies and Systems (USITS)*. [S.l.: s.n.], 2003. p. 1–16.

RUIJTERS, E.; STOELINGA, M. Fault tree analysis: A survey of the state-of-the-art in modeling, analysis and tools. *Computer Science Review*, v. 15, p. 29–62, 2015.

SCHROEDER, B.; GIBSON, G. A. A large-scale study of failures in high-performance computing systems. In: *Proceedings of the International Conference on Dependable Systems and Networks (DSN)*. Philadelphia, PA, USA: [s.n.], 2006. p. 249–258.

TIAN, J.; RUDRARAJU, S.; LI, Z. Evaluating web software reliability based on workload and failure data extracted from server logs. *IEEE Transactions on Software Engineering*, v. 30, n. 11, p. 754–769, 2004.